



CSC232 - INFORMATION SECURITY

Lab Assignment 04 Report



Submitted By:

Muhammad Rizwan Shafiq
SP24-BCS-069
BCS-4B

Submitted To:

Sir Sabghat Ullah

Table of Contents

1. System Architecture	3
1.1 Overall System Flow	3
1.2 Communication Protocol.....	3
Setup Phase	3
Message Transmission.....	3
Reception & Verification	3
Task 1: RSA Implementation	4
Key Code Snippet: RSA Key Generation	4
Encryption / Decryption Snippet.....	4
Outcome	4
Task 2: PKI Certificate Management (Mock CA).....	4
Key Code Snippet: Certificate Issuance	4
CRL Snippet.....	5
Outcome	5
Task 3: Digital Signature Algorithm (DSA)	5
Features	5
Testing.....	5
Non-repudiation	5
Key Code Snippet: Signing	5
Verification Snippet.....	6
Outcome	6
Task 4: Lamport One-Time Signature (OTS).....	6
Key Code Snippet: Key Generation	6
Signing Snippet	6
Outcome	6
Comparison with DSA	6
Task 5: Integrated Secure Communication System	7
Key Snippet: Encryption + Signing Workflow	7

Verification Snippet	7
Important Components	7
Scenario 1: Successful Communication	7
Scenario 2: Message Tampering Detection	7
Scenario 3: Certificate Revocation	8
Outcome Summary	8
Conclusion	8

Public Key Infrastructure, Public Key Cryptography, Digital Signature Algorithm & One-Time Signatures

1. System Architecture

This project integrates multiple cryptographic components including RSA, DSA, Lamport OTS, and a Mock Certificate Authority to build a complete secure communication system.

1.1 Overall System Flow

The following steps outline the architecture:

1. **Key Generation** (RSA / DSA / OTS)
2. Certificate Creation by a Mock CA
3. Message Encryption using RSA (PKC)
4. **Message Signing** using DSA and Lamport OTS
5. Verification & Decryption by the receiving party

This ensures confidentiality, authentication, and integrity.

1.2 Communication Protocol

The communication protocol follows a structured three-phase process:

Setup Phase

- Alice and Bob generate RSA key pairs.
- Mock CA issues certificates for both.
- Certificates are exchanged.

Message Transmission

- Alice writes message **M**.
- RSA encryption: **C = Enc_Bob(M)**.
- DSA signature: **$\sigma_{\text{DSA}} = \text{Sign_Alice}(M)$** .
- OTS signature: **$\sigma_{\text{OTS}} = \text{OTS_Sign}(M)$** .
- Alice sends $(C, \sigma_{\text{DSA}}, \sigma_{\text{OTS}}, \text{Cert_Alice})$.

Reception & Verification

- Bob verifies Alice's certificate with the CA.
- Bob verifies both signatures.
- Bob decrypts the ciphertext to recover message **M**.

- The system ensures **privacy, authentication, and integrity**.

Task 1: RSA Implementation

The RSA module provides functionality for generating keys, encrypting, and decrypting messages.

Key Code Snippet: RSA Key Generation

```
# From Task_01_RSA_Implementation.py
def generate_rsa_keys(keysize=1024):
    half_bits = keysize // 2
    p = generateLargePrime(half_bits)
    q = generateLargePrime(half_bits)
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    d = mod_inverse(e, phi)
    return (e, n), (d, n)
```

Encryption / Decryption Snippet

```
def rsa_encrypt(message, public_key):
    e, n = public_key
    m_int = int.from_bytes(message.encode('utf-8'), 'big')
    return pow(m_int, e, n)

def rsa_decrypt(ciphertext, private_key):
    d, n = private_key
    m_int = pow(ciphertext, d, n)
    return m_int.to_bytes((m_int.bit_length()+7)//8, 'big').decode()
```

Outcome

Three messages were encrypted and decrypted correctly.

Task 2: PKI Certificate Management (Mock CA)

The Mock CA simulates essential PKI functionality.

Key Code Snippet: Certificate Issuance

```
# From Task_02_Mock_CA.py
cert = (
    x509.CertificateBuilder()
    .subject_name(subject)
    .issuer_name(self.ca_certificate.subject)
    .public_key(subject_private_key.public_key())
```

```
.serial_number(x509.random_serial_number())
.not_valid_before(now)
.not_valid_after(now + timedelta(days=validity_days))
.add_extension(x509.BasicConstraints(ca=False), critical=True)
)
certificate = cert.sign(private_key=self.ca_private_key, algorithm=hashes.SHA
256())
```

CRL Snippet

```
revoked_cert = (
    x509.RevokedCertificateBuilder()
    .serial_number(serial)
    .revocation_date(datetime.utcnow())
    .add_extension(x509.CRLReason(x509.ReasonFlags.key_compromise), critical=
False)
    .build()
)
```

Outcome

Certificates are generated, validated, and revoked correctly.

Task 3: Digital Signature Algorithm (DSA)

DSA was implemented from scratch, including prime generation and signature mechanics.

Features

- Prime generation using Miller–Rabin.
- Generator computation and validation.
- Signature pair creation (c1, c2).
- SHA-1 hashing for message digest.
- Signature verification by reconstructing hash and public parameters.

Testing

- Verified signatures for valid messages.
- Invalid messages correctly detected.

Non-repudiation

Since the private key is only known to the signer, DSA confirms message authorship.

Key Code Snippet: Signing

```
# From sign.py
c1 = squareAndMultiply(g, r, p) % q
c2 = shaHash(fileName) + (a * c1)
```

```
Rinverse = computeInverse(r, q)
c2 = (c2 * Rinverse) % q
```

Verification Snippet

```
valid1 = squareAndMultiply(g, t1, p)
valid2 = squareAndMultiply(h, t2, p)
valid = ((valid1 * valid2) % p) % q
```

Outcome

Valid messages verify successfully; tampered messages fail.

Task 4: Lamport One-Time Signature (OTS)

A hash-based signature scheme was implemented.

Key Code Snippet: Key Generation

```
# From Lamport_One_Time_Sign.py
for i in range(255):
    skey[k][i] = bin(secrets.randbits(255))[2:].zfill(255)
    pkey[k][i] = hashlib.sha256(skey[k][i].encode()).hexdigest()
```

Signing Snippet

```
mhash = int(hashlib.sha256(message.encode()).hexdigest(), 16)
for i in range(255):
    k = (mhash >> i) & 1
    signature[i] = skey[k][i]
```

Outcome

OTS signatures validated correctly, reuse breaks security.

Comparison with DSA

Feature	DSA	Lamport OTS
Type	Number-theoretic	Hash-based
Security	Relies on discrete log	Relies purely on hashes
Reusability	Multiple use	One-time only
Efficiency	Faster keys, small signatures	Large signatures, slow

Task 5: Integrated Secure Communication System

This system integrates RSA, DSA, OTS, and PKI end-to-end.

Key Snippet: Encryption + Signing Workflow

```
# Encrypt message with Bob's RSA key
C = rsa_encrypt(M, bob_pub)

# DSA signing
msg_file.write_text(M)
dsa_sign_file('task5_message.txt')

# OTS signing
ots_signature = ots_sign(M, ots_skey)
```

Verification Snippet

```
# Verify certificate
ca.verify_certificate(alice_cert)

# Verify DSA
dsa_verify_file('task5_message.txt')

# Verify OTS
ots_verify(M, ots_pkey, ots_signature)
```

Important Components

- RSA: Confidentiality
- DSA: Signature-based authentication
- OTS: Additional integrity verification
- CA: Trust management

Scenario 1: Successful Communication

- Alice encrypts message with Bob's RSA public key.
- Alice signs message using DSA and OTS.
- Bob validates certificate, signatures, and decrypts message.
- Output confirms successful, authenticated communication.

Scenario 2: Message Tampering Detection

- Attacker modifies message.
- DSA verification fails.

- Lamport OTS verification fails.
- System detects tampering.

Scenario 3: Certificate Revocation

- CA revokes Alice's certificate.
- Bob attempts to verify—fails.
- System stops communication even with valid signatures.

Outcome Summary

- Successful communication scenario works perfectly.
- Tampering is detected by both DSA and OTS.
- Certificate revocation halts trust even when signatures are correct.

The integrated system provides:

- Confidentiality (RSA)
- Authentication (DSA, OTS, Certificates)
- Integrity (DSA & OTS)
- Trust Management (CA + CRL)

Conclusion

This assignment successfully demonstrates:

- Complete lifecycle of public key cryptography.
- How digital signatures ensure trust and message integrity.
- How PKI enforces authentication and revocation.
- A fully functional secure communication model.

The combined implementation provides a realistic educational demonstration of modern cryptographic systems.